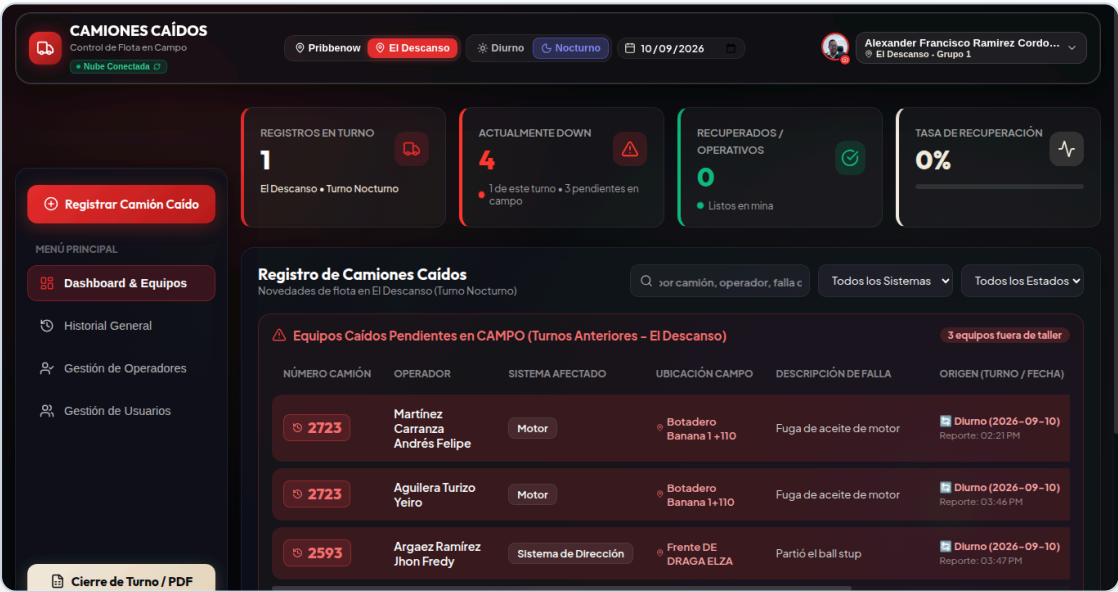


DOCUMENTACIÓN DE INGENIERÍA

CAMIONES CAÍDOS

MANUAL TÉCNICO DE ARQUITECTURA, DESPLIEGUE Y RECUPERACIÓN

Guía para Ingenieros de Software, DevOps y Soporte L3: Arquitectura SPA, GoTrue, Row Level Security, Edge Functions, PWA Workbox y Runbooks de Contingencia



ÍNDICE DE SECCIONES TÉCNICAS

1. Resumen de Arquitectura Decoupled SPA y Serverless Edge	2. Stack y Versiones React 19, Vite 8, Supabase JS 2.109
3. Estructura del Proyecto Árbol de módulos, componentes y lib	4. Flujo de Autenticación GoTrue, derivación técnica y tokens
5. Modelo de Datos DDL Esquema relacional de tablas y campos	6. Relaciones de Datos Vínculos de auth.users a perfiles
7. operator_id y Snapshot Diseño relacional dual resiliente	8. Row Level Security 11 políticas y trigger de inmutabilidad
9. Edge Functions Deno admin-create, reset y delete-user	10. Sesión Offline Tolerancia a fallas y online fallback
11. Almacenamiento Local Claves seguras en localStorage	12. Motor PWA Manifiesto web y capacidades de campo
13. Workbox Service Worker Precache de 26 activos y runtime cache	14. Lazy Loading React.lazy en módulos pesados
15. manualChunks Rollup Segmentación granular de bundles	16. Exportación PDF/Excel jsPDF y xlsx en cliente
17. Variables de Entorno Nombres oficiales de frontend y cloud	18. Desarrollo Local Comandos npm y servidores locales
19. Pipeline de Compilación Vite build y métricas de empaquetado	20. Análisis Estático oxlint: 0 errores, 28 warnings baseline
21. Despliegue en Vercel Configuración vercel.json y rewrites	22. Infraestructura Supabase PostgreSQL 15, pooler y Deno Edge
23. Estrategia de Respaldos Código, base de datos y configuración	24. Runbooks de Recuperación Protocolos A a G ante contingencias
25. Seguridad en Profundidad Aislamiento de service_role y RLS	26. Métricas de Performance Bundle inicial 125 kB, gzip 33.5 kB
27. Snapshot del Release 22 usuarios, 814 operadores, 347 reportes	28. Archivos Sensibles Módulos protegidos y criticidad
29. Troubleshooting Matriz de resolución de fallas técnicas	30. Checklist Mantenimiento Rutina de salud pre y post-release

1. RESUMEN DE ARQUITECTURA

La solución Camiones Caídos implementa una arquitectura desacoplada Single Page Application (SPA) orientada a la máxima resiliencia en terreno minero:

- **Frontend SPA:** Construido con React 19 y empaquetado con Vite 8. Aloja la lógica de presentación, cálculo de KPIs en memoria y renderizado gráfico adaptativo. Se distribuye mediante la CDN de Vercel con integración PWA para trabajo offline en tajos.
- **Backend as a Service (BaaS):** Supabase Cloud proporciona el motor PostgreSQL 15, el subsistema de autenticación GoTrue y la seguridad declarativa en capa de datos mediante Row Level Security (RLS).
- **Capa Serverless Edge:** Tres Edge Functions en Deno TypeScript asumen la ejecución de operaciones privilegiadas de administración de usuarios y claves bajo el principio de menor privilegio.

2. STACK TECNOLÓGICO Y VERSIONES VERIFICADAS

Tecnología / Paquete	Versión Verificada	Rol Arquitectónico
React / React DOM	19.2.8	Librería base de componentes y renderizado reactivo
Vite	8.2.0	Servidor de desarrollo y bundler de producción (Rollup 4)
@supabase/supabase-js	2.109.0	SDK cliente para conexión a PostgreSQL, Auth y Storage
vite-plugin-pwa / Workbox	1.3.0 / v7	Generación de Service Worker y precaching offline
jspdf / jspdf-autotable	4.2.1 / 5.0.8	Motor cliente para exportación ejecutiva de informes PDF
xlsx	0.18.5	Generación de hojas de cálculo de doble pestaña en navegador
oxlint	1.75.0	Linter estático de alto rendimiento en Rust (0 errores)
Supabase Cloud	PostgreSQL 15+	Motor de base de datos relacional y servidor GoTrue
Vercel	Serverless Edge	Alojamiento del frontend y entrega HTTPS global con rewrites SPA

3. ESTRUCTURA DEL PROYECTO

```
camiones-caidos-app/
├─ package.json / vite.config.js / vercel.json
├─ supabase_schema.sql      # DDL oficial, RLS, triggers y funciones helper
├─ supabase/functions/      # Edge Functions en Deno TypeScript
│   └─ admin-create-user/    # Aprovisionamiento atómico
│   └─ admin-reset-password/ # Reseteo a clave temporal con backoff
│   └─ admin-delete-user/    # Purga coordinada con protecciones
├─ src/
│   └─ main.jsx / App.jsx / index.css
│   └─ lib/supabase.js       # Cliente singleton @supabase/supabase-js
│   └─ context/
│       └─ AuthContext.jsx    # GoTrue, derivación técnica y offline fallback
│       └─ ReportContext.jsx  # Estado de reportes, operadores y caché
│   └─ components/
│       └─ Layout/            # Navbar, MobileNav.jsx, LiquidLensCanvas.jsx
│       └─ Management/        # UserManager.jsx, OperatorManager.jsx
│       └─ Forms/             # TruckReportModal.jsx (captura y edición)
│       └─ Reports/           # GlobalHistory.jsx, TruckHistoryModal.jsx
│       └─ Dashboard/         # FleetMetrics, StatusCards, SystemBreakdown
│   └─ utils/                 # pdfUtils.js, exportUtils.js, dateUtils.js
```

4. FLUJO DE AUTENTICACIÓN

La autenticación desacopla las credenciales de los datos operacionales de aplicación:

- `auth.users` (GoTrue): Custodia contraseñas mediante hashing seguro y emite tokens JWT firmados.
- `public.app_users`: Almacena perfil, rol, sede, grupo y la bandera `must_change_password`, enlazados mediante `auth_user_id` (UUID).
- **Derivación de Email Técnico:** En el login, el usuario ingresa su cédula. El sistema deriva el email canónico determinista:

```
const nationalIdToTechnicalEmail = (id) => `${cleanNationalId(id)}@camionescaidos.internal`;
```

- **Sesión:** Manejada por Supabase SDK con refresco automático de token.

5. MODELO DE DATOS RELACIONAL Y ESQUEMA DDL

```
-- Perfiles de aplicación enlazados a auth.users
CREATE TABLE public.app_users (
  id TEXT PRIMARY KEY,
  national_id TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT 'Encargado',
  mine TEXT NOT NULL DEFAULT 'Pribbenow',
  group_name TEXT NOT NULL DEFAULT 'Grupo 1',
  must_change_password BOOLEAN DEFAULT TRUE,
  avatar TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  auth_user_id UUID UNIQUE
);

-- Censo centralizado de operadores
CREATE TABLE public.operators (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  mine TEXT NOT NULL DEFAULT 'Pribbenow',
  group_name TEXT NOT NULL DEFAULT 'Grupo 1',
  status TEXT DEFAULT 'Activo',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Registro de novedades de camiones
CREATE TABLE public.truck_reports (
  id TEXT PRIMARY KEY,
  truck_id TEXT NOT NULL,
  mine TEXT NOT NULL,
  shift TEXT NOT NULL DEFAULT 'Diurno',
  operator TEXT,                -- Snapshot textual histórico
  system TEXT,
  detail TEXT,
  location TEXT,
  status TEXT DEFAULT 'DOWN',
  down_time TEXT,
  estimated_return_time TEXT,
  actual_return_time TEXT,
  date TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  operator_id TEXT              -- Clave foránea lógica a operators.id
);
```

6. ARQUITECTURA RELACIONAL DUAL (operator_id + operator)

La tabla `truck_reports` implementa una arquitectura relacional dual intencional:

- `operator_id`: Clave foránea lógica que referencia a `operators.id`. Facilita análisis de recurrencia, cruces estadísticos y normalización.
- `operator`: Snapshot textual que captura la cadena literal del nombre al momento del reporte.
- **Razón de Diseño**: Proteger la inmutabilidad histórica. Si un operador es editado, corregido o dado de baja del censo, los reportes históricos conservan el valor original exacto. Adicionalmente, permite generar informes PDF/Excel en modo offline sin realizar consultas de join.

7. ROW LEVEL SECURITY (RLS) Y FUNCIONES HELPER

Row Level Security está habilitado en las tres tablas mediante 11 políticas activas y 3 funciones helper `SECURITY DEFINER` con `search_path = 'public':`

```
CREATE FUNCTION public.get_auth_user_role() RETURNS text LANGUAGE sql STABLE SECURITY DEFINER SET search_path TO
'public' AS $$
  SELECT role FROM public.app_users WHERE auth_user_id = auth.uid() LIMIT 1;
$$;

CREATE FUNCTION public.get_auth_user_mine() RETURNS text LANGUAGE sql STABLE SECURITY DEFINER SET search_path TO
'public' AS $$
  SELECT mine FROM public.app_users WHERE auth_user_id = auth.uid() LIMIT 1;
$$;

CREATE FUNCTION public.is_admin() RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER SET search_path TO
'public' AS $$
  SELECT EXISTS (SELECT 1 FROM public.app_users WHERE auth_user_id = auth.uid() AND role = 'Administrador');
$$;
```

Trigger de Inmutabilidad: La función `trg_protect_app_users_columns()` asociada a `BEFORE UPDATE ON public.app_users` prohíbe la alteración de `id`, `national_id`, `auth_user_id` y `created_at`, garantizando blindaje contra elevación no autorizada de privilegios.

8. SUPABASE EDGE FUNCTIONS (SERVERLESS DENO)

- **admin-create-user:** Valida rol Administrador en DB, chequea duplicados (409 Conflict), crea usuario en Auth con clave temporal `caidos1234` e inserta en `app_users` con `must_change_password: true`. Implementa rollback compensatorio purgando la cuenta de Auth si falla la base de datos.
- **admin-reset-password:** Restablece la clave a `caidos1234` en Auth y actualiza `must_change_password = true` con hasta 3 reintentos de retroceso exponencial.
- **admin-delete-user:** Bloquea autoeliminación (400 Bad Request) y protege al último administrador del sistema. Ejecuta eliminación atómica coordinada en Auth y base de datos.

9. GESTIÓN DE SESIÓN OFFLINE Y LOCALSTORAGE

`AuthContext` maneja la resiliencia en terreno minero:

- Si falla el transporte de red, recupera el perfil validado de `localStorage['camiones_user_profile']` y mantiene la UI activa en modo lectura/consulta.
- Claves autorizadas en `localStorage`: `camiones_user_profile`, `camiones_reports`, `camiones_operators`, `camiones_mine` y el token de sesión emitido por GoTrue. Ninguna contraseña en texto plano se almacena.

10. MOTOR PWA Y SERVICE WORKER (WORKBOX 7)

Generado con `vite-plugin-pwa`:

- Service Worker: `dist/sw.js` en modo `autoUpdate`.
- Precaching: 26 entradas estáticas (2,314.69 KiB) cacheadas obligatoriamente en la instalación.
- Soporte Offline: Permite consultar el Dashboard y la bitácora histórica sin cobertura celular.

11. OPTIMIZACIÓN MANUALCHUNKS Y RENDIMIENTO

Vite 8 empaqueta los módulos de exportación y librerías pesadas en fragmentos diferidos (`React.lazy`):

- `vendor-export-pdf` (~657 kB): `jspdf`, `jspdf-autotable`, `html2canvas`.
- `vendor-export-xlsx` (~282 kB): `xlsx` y códecs binarios.
- `vendor-supabase` (~203 kB): SDK cliente de Supabase.
- `vendor-react` (~189 kB): Núcleo de React y ReactDOM.
- **Métricas de Build Certificadas:** Bundle inicial: **125.83 kB** (33.55 kB gzip). Hoja de estilos CSS: **15.81 kB** (3.78 kB gzip).

12. DESARROLLO, BUILD, LINT Y DESPLIEGUE EN VERCEL

```
# Desarrollo local
npm run dev

# Compilación de producción (genera carpeta dist/)
npm run build

# Linter estático (oxlint: 0 errores, 28 warnings baseline)
npm run lint

# Previsualización productiva local
npm run preview
```

Configuración Vercel (vercel.json):

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Variables de Entorno (Solo nombres requeridos):

- Frontend (Vercel): VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY.
- Backend (Supabase Secrets): SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY.

13. PLAN DE RESPALDOS (BACKUPS)

- **Código:** Repositorio Git centralizado en GitHub (rama main sincronizada).
- **Configuración y DDL:** supabase_schema.sql y código de Edge Functions versionados en el repositorio.
- **Datos:** Snapshots JSON auditados pre-migración (truck_reports_pre_operator_id_migration_2026-09-10.json) y copias de seguridad automáticas diarias gestionadas por la infraestructura Supabase Cloud. No existe cron de respaldo dentro del frontend.

14. RUNBOOKS DE RECUPERACIÓN ANTE INCIDENTES

Runbook	Síntoma	Diagnóstico	Acción Correctiva	Verificación
A. Frontend no carga	Pantalla blanca con error ChunkLoadError.	Service Worker solicita chunks con hashes antiguos ya purgados en Vercel.	Instruir al usuario a presionar Ctrl + F5 o limpiar datos de navegación del sitio.	Carga exitosa del Dashboard en navegador limpio.
B. Deploy fallido	Error de build en Vercel CI/CD.	Fallo de empaquetado o variables ausentes.	Ejecutar npm run build localmente; validar variables en Vercel. Si persiste, aplicar Instant Rollback en Vercel.	Despliegue con estado READY en Vercel.
C. Supabase caído	Errores de red en peticiones REST.	Indisponibilidad del backend o límite de conexiones del pooler.	Verificar status.supabase.com. La app conmuta a modo offline. Si el pooler está saturado, reiniciar servicio en panel.	Retorno de código 200 en /rest/v1/truck_reports.
D. JWT expirado	HTTP 401 Unauthorized en peticiones.	Token de sesión caducado o revocado en GoTrue.	AuthContext captura 401 y ejecuta logout(). El usuario debe reingresar credenciales en LoginForm.	Emisión de nuevo token de sesión válido.
E. Problema de RLS	Consultas vacías o error 403 Forbidden.	Perfil desalineado en app_users o función is_admin() denegada.	Verificar que auth_user_id en app_users coincida con auth.uid().	Visualización correcta de datos según rol y mina.
F. Caché corrupta	Datos antiguos persisten tras actualización.	Conflictos en IndexedDB o Service Worker no reactivado.	Borrar almacenamiento del sitio en navegador y desregistrar Service Worker.	Descarga fresca del censo y reportes activos.
G. Corrupción de datos	Pérdida de registros en truck_reports.	Borrado no autorizado o script anómalo.	REQUIERE PROCEDIMIENTO ADICIONAL: Restaurar desde snapshot JSON pre-migración o backup diario de Supabase.	Conciliación de integridad relacional al 100%.

15. SEGURIDAD, PERFORMANCE Y SNAPSHOT DE RELEASE

- **Seguridad:** El cliente web solo utiliza anon key; la llave service_role está blindada en servidores Deno. El trigger trg_protect_app_users_columns asegura la inmutabilidad de la identidad.
- **Performance Certificada:** Bundle inicial: **125.83 kB** (33.55 kB gzipped). CSS inicial: **15.81 kB** (3.78 kB gzipped). Precache: 26 activos (2.31 MB).
- **Snapshot de Datos de Release:**
 - app_users = 22 usuarios.
 - operators = 814 conductores (100% en estado 'Activo').
 - truck_reports = 347 registros totales.
 - operator_id NOT NULL = 324 (296 históricos migrados + 28 creados en producción).
 - operator_id NULL = 23 (18 "Sin Registro" y 5 "Operador Modificado"; excepciones históricas documentadas).

16. INVENTARIO DE ARCHIVOS SENSIBLES Y CRÍTICOS

- AuthContext.jsx: Núcleo de autenticación, gestión de tokens JWT y tolerancia a fallas offline.
- ReportContext.jsx: Estado central de reportes, catálogo de operadores y sincronización con Supabase.
- MobileNav.jsx y LiquidLensCanvas.jsx: Motor visual y táctil de navegación móvil.
- TruckReportModal.jsx: Formulario maestro de captura de fallas de camiones.
- pdfUtils.js: Generador de informes ejecutivos en PDF para gerencia.
- supabase/functions/: Edge Functions custodias de la gobernanza de usuarios y contraseñas.

17. CHECKLIST DE MANTENIMIENTO TÉCNICO

1. **Semanal:** Monitorear métricas y logs de Edge Functions en la consola de Supabase.
2. **Mensual:** Ejecutar `npm run lint` y `npm audit` para verificar la sanidad del árbol de dependencias.
3. **Procedimiento Pre-Release Obligatorio (9 Pasos):** 1. Git limpio; 2. `npm run lint` (0 errores); 3. `npm run build` (exit code 0); 4. Verificar variables en Vercel; 5. Deploy; 6. Comprobar Supabase REST y RLS; 7. Smoke test en producción; 8. Comprobar PWA Service Worker; 9. Verificar rollback.